

ASP.NET Core provides a set of built-in route constraints, such as the following:

- `int`: The parameter must be an integer value.
- `bool`: The parameter must be a Boolean value.
- `datetime`: The parameter must be a `DateTime` value.
- `decimal`: The parameter must be a decimal value. Similarly, there are `double`, `float`, `long`, and so on.
- `guid`: The parameter must be a GUID value.
- `minlength(value)`: The parameter must be a string with a minimum length; for example, `{name:minlength(6)}`, which means the `name` parameter must be a string and the length of the string must be at least 6 characters. Similarly, there are `maxlength(value)`, `length(value)`, `length(min, max)`, and so on.
- `min(value)`: The parameter must be an integer with a minimum value; for example, `{id:min(1)}`, which means the `id` parameter must be an integer and the value must be greater than or equal to 1. Similarly, there are `max(value)`, `range(min, max)`, and so on.
- `alpha`: The parameter must be a string with one or more letters.
- `regex(expression)`: The parameter must be a string that matches the regular expression.
- `required`: The parameter must be provided in the route; for example, `{id:required}`, which means the `id` parameter must be provided in the route.

If the value of the route parameter does not match the route constraint, *the action method will not accept the request*, and a *404 Not Found response* will be returned.

Multiple route constraints can be applied together. The following code shows how to apply multiple route constraints to the `id` parameter, which means the `id` parameter must be an integer and the value must be greater than or equal to 1 and less than or equal to 100:

```
[HttpGet("{id:int:range(1, 100)}")]
public async Task<ActionResult<Post>> GetPost(int id)
{
    // Omitted for brevity
}
```

Route constraints can be used to make a route more specific. However, they should not be used to validate the input. If the input is invalid, the API should return a `400 Bad Request` response rather than a `404 Not Found` response.

ASP.NET Core offers the following binding source attributes:

- `[FromBody]`: The parameter is from the request body
- `[FromForm]`: The parameter is from the form data in the request body
- `[FromHeader]`: The parameter is from the request header
- `[FromQuery]`: The parameter is from the query strings in the request
- `[FromRoute]`: The parameter is from the route path
- `[FromServices]`: The parameter is from the **DI** container

For example, we can define a pagination action method as follows:

```
[HttpGet("paged")]
public async Task<ActionResult<List<Post>>> GetPosts([FromQuery] int
pageIndex, [FromQuery] int pageSize)
{
    // Omitted for brevity
}
```

When an `[ApiController]` attribute is applied to a controller class, a set of default inference rules will be applied, so we do not need to explicitly add these binding source attributes. For example, the following code shows a POST action method:

```
[HttpPost]
public async Task<ActionResult<Post>> CreatePost(Post post)
{
    // Omitted for brevity
}
The post parameter is a complex type, so [FromBody] inferred that
the post should be from the request body. But [FromBody] is not
inferred for simple data types, such as int, string, and so on. We
will define an action method as follows:
[HttpPost("search")]
public async Task<ActionResult<Post>> SearchPosts(string keyword)
```

The keyword parameter is a simple type, so `[FromQuery]` inferred that the keyword parameter should be from the query strings in the URL, such as `/api/posts/search?keyword=xyz`. If we want to force the keyword parameter to be from the request body, we can use the `[FromBody]` attribute as follows:

```
[HttpPost("search")]
public async Task<ActionResult<Post>> SearchPosts([FromBody] string
keyword)
```

The default inference rules of those binding source attributes are listed next:

- For complex type parameters, if the type is registered in the DI container, `[FromServices]` is inferred.
- For complex type parameters that are not registered in the DI container, `[FromBody]` is inferred. It does not support multiple `[FromBody]` parameters.
- For types such as `IFormFile` and `IFormFileCollection`, `[FromForm]` is inferred.
- For any parameters that appear in the route, `[FromRoute]` is inferred.

ASP.NET Core provides a comprehensive configuration framework that makes it easy to work with configuration settings. A configuration is considered a key-value pair. These configuration settings are stored in a variety of sources, such as JSON files, environment variables, and command-line arguments, or in the cloud, such as Azure Key Vault. In ASP.NET Core, these sources are referred to as configuration providers. Each configuration provider is responsible for loading configuration settings from a specific source.

ASP.NET Core supports a set of configuration providers, such as the following:

- The file configuration provider, such as, `appsettings.json`
- The User secrets
- The environment variables configuration provider
- The command-line configuration provider
- The Azure App Configuration provider
- The Azure Key Vault configuration provider

You will find another `appsettings.Development.json` file, which will be used for the development environment. We will introduce the environment in the following section.

Let us add a `"MyKey": "MyValue"` key-value pair to the `appsettings.json` file. This key-value pair is an example configuration that we will read in the code using `JsonConfigurationProvider`:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "MyKey": "MyValue"
}
```

Create a new controller named `ConfigurationController` in the `Controllers` folder. In this controller, we will read the configuration value from the `appsettings.json` file and return it as a string. The following code shows the `ConfigurationController` class:

```

public class ConfigurationController(IConfiguration configuration) :
    ControllerBase
{
    [HttpGet]
    [Route("my-key")]
    public ActionResult GetMyKey()
    {
        var myKey = configuration["MyKey"];
        return Ok(myKey);
    }
}

```

The configuration supports hierarchical settings. For example, consider the following configuration settings:

```

{
  "Database": {
    "Type": "SQL Server",
    "ConnectionString": "This is the database connection string"
  }
}

```

To access the Type and ConnectionString properties, we can use the following code:

```

[HttpGet]
[Route("database-configuration")]
public ActionResult GetDatabaseConfiguration()
{
    var type = configuration["Database:Type"];
    var connectionString = configuration["Database:ConnectionString"];
    return Ok(new { Type = type, ConnectionString = connectionString
});
}

```

Using the *IConfiguration interface*, we can access the configuration settings with the configuration[key] format. However, hardcoding the keys is not a good practice. To avoid hardcoding, ASP.NET Core supports the options pattern, which can provide a strongly typed way to access hierarchical settings.

Using the options pattern

To use the options pattern, we need to create a class that represents the configuration settings. The following code shows how to create a class named DatabaseOption:

```
namespace ConfigurationDemo;

public class DatabaseOption
```

Configuration

```
{
    public const string SectionName = "Database";
    public string Type { get; set; } = string.Empty;
    public string ConnectionString { get; set; } = string.Empty;
}
```

Using the *ConfigurationBinder.Get<TOption>()* method

We can also use the `ConfigurationBinder.Get<TOption>()` method, which attempts to bind the configuration instance to a new instance of type `T`. If this configuration section has a value, then that value will be used; otherwise, it attempts to bind the configuration instance by matching property names against configuration keys recursively. The code is shown next:

```
[HttpGet]
[Route("database-configuration-with-generic-type")]
public ActionResult GetDatabaseConfigurationWithGenericType()
{
    var databaseOption = configuration.GetSection(DatabaseOption.
        SectionName).Get<DatabaseOption>();
    return Ok(new { databaseOption.Type, databaseOption.
        ConnectionString });
}
```

Run the application and send a request to the `/Configuration/database-configuration-with-generic-type` endpoint. You will see the same response as in the *Using appsettings.json* section.

Using the *IOptions<TOption>* interface

ASP.NET Core provides built-in DI support for the options pattern. To use DI, we need to register the `DatabaseOption` class in the `Services.Configure()` method of the `Program.cs` file. The following code shows how to register the `DatabaseOption` class:

```
// Register the DatabaseOption class as a configuration object.
// This line must be added before the `builder.Build()` method.
builder.Services.Configure<DatabaseOption>(builder.Configuration.
    GetSection(DatabaseOption.SectionName));

var app = builder.Build();
```

Next, we can use DI to inject the `IOptions<DatabaseOption>` interface into the `ConfigurationController` class. The following code shows how to inject the `IOptions<DatabaseOption>` interface:

```
[HttpGet]
[Route("database-configuration-with-ioptions")]
public ActionResult
    GetDatabaseConfigurationWithIOptions([FromServices]
        IOptions<DatabaseOption> options)
{
    var databaseOption = options.Value;
```

```
        return Ok(new { databaseOption.Type, databaseOption.
ConnectionString });
    }
}
```

Let's try to use the `IOptionsSnapshot<T>` interface to replace the `IOptions<TOption>` interface. The `IOptionsSnapshot<TOption>` interface provides a snapshot of options for the current request. The following code shows how to use the `IOptionsSnapshot<TOption>` interface:

```
[HttpGet]
[Route("database-configuration-with-ioptions-snapshot")]
public ActionResult
GetDatabaseConfigurationWithIOptionsSnapshot([FromServices]
IOptionsSnapshot<DatabaseOption> options)
{
    var databaseOption = options.Value;
    return Ok(new { databaseOption.Type, databaseOption.
ConnectionString });
}
```

Run the application again. Change the `Type` property in the `appsettings.json` file. Send requests to the `/Configuration/database-configuration-with-ioptions-snapshot` endpoint. You will find the response is the *new* value.

Okay, we now know the difference between the `IOptions<TOption>` interface and the `IOptionsSnapshot<TOption>` interface:

- The `IOptions<TOption>` interface provides a way to access options, but it cannot get the latest value if the setting value has been changed when the application is running.
- The `IOptionsSnapshot<TOption>` interface provides a snapshot of options for the current request. The `IOptionsSnapshot<TOption>` interface is useful when we want to get the latest options for the current request.

But why?

The ASP.NET Core framework provides built-in support for `appsettings.json` using `JsonConfigurationProvider`, which reads the configuration values from the `appsettings.json` file. When the framework registers `JsonConfigurationProvider`, the code looks like this:

```
config.AddJsonFile("appsettings.json", optional: true, reloadOnChange: true)
    .AddJsonFile($"appsettings.{env.EnvironmentName}.json", optional: true, reloadOnChange: true);
```

The `reloadOnChange` parameter is set to `true`, which means the configuration values will be reloaded if the `appsettings.json` file has been changed. So, the `ConfigurationBinder.Bind()` method and the `ConfigurationBinder.Get<TOption>()` method can get the latest value.

However, when the `IOptions<TOption>` interface is registered by the ASP.NET Core framework, it is registered as a *singleton* service, which means an instance of `IOption<TOption>` will be created only once. You can inject it into any service lifetime, but it cannot read the latest value if the setting value has been changed.

In contrast, the `IOptionsSnapshot<TOption>` interface is registered as a *scoped* service, so it cannot be injected into a singleton service. It is useful in scenarios if you want to get the latest options for each request.

There is another options interface called `IOptionsMonitor<TOption>`. It is a combination of the `IOptions<TOption>` and the `IOptionsSnapshot<TOption>` interfaces. It provides the following features:

- It is a singleton service, which can be injected into any service lifetime
- It supports reloadable configuration

Here is an example of using the `IOptionsMonitor<TOption>` interface:

```
[HttpGet]
[Route("database-configuration-with-ioptions-monitor")]
public ActionResult
GetDatabaseConfigurationWithIOptionsMonitor([FromServices]
IOptionsMonitor<DatabaseOption> options)
{
    var databaseOption = options.CurrentValue;
    return Ok(new { databaseOption.Type, databaseOption.
ConnectionString });
}
```

The `IOptionsMonitor<TOption>` interface provides the `CurrentValue` property to get the latest value. It also provides the `OnChange(Action<TOption, string> listener)` method to register a listener that will be called whenever the options are reloaded. Normally, you do not need to use the `OnChange()` method unless you want to do something when the options are reloaded.
